



INSTITUTO TECNOLÓGICO SUPERIOR DE LA
REGIÓN DE LOS LLANOS

INGENIERÍA MECATRÓNICA

PROGRAMACIÓN AVANZADA

Actividad: U3A2. ETAPA 2. Integración de Hardware

Unidad 3

Alumnos:

Alejandra Berenice Flores García

Esmeralda Gómez Huerta

Jean Paul Acosta Navarro

Eber Jafet Rodriguez Valenciano

Docente: Osbaldo Aragon Banderas

Fecha: 2026-05-09

1. Objetivo

Que el alumno sea capaz de seleccionar, conectar y programar el hardware necesario —microcontroladores, sensores y actuadores— para el proyecto DriverWatch, desarrollando el software embebido que permita la adquisición de datos, el control de dispositivos y la comunicación con otros sistemas y plataformas.

En el contexto del proyecto, esto implica integrar físicamente el módulo de visión artificial (Python + MediaPipe), el microcontrolador ESP32-C3 con su firmware de máquina de estados finita (C++/Arduino), el broker MQTT HiveMQ, la plataforma Node-RED y la base de datos SQLite, formando un pipeline funcional que detecta y alerta sobre estados de fatiga del conductor en tiempo real.

2. Descripción General

Con los requerimientos definidos en la Etapa 1, la Etapa 2 da vida física al sistema DriverWatch. Se identificó con precisión qué hardware se requiere, cómo interactúa con el software y qué canales de comunicación conectan cada capa del sistema. Esta fase es crítica en proyectos mecatrónicos porque une el mundo físico (cámara, buzzer, señales GPIO) con el mundo digital (landmarks faciales, topics MQTT, registros SQLite).

La actividad se estructura en cuatro fases complementarias:

- Fase 1 — Selección y documentación del hardware del sistema.
- Fase 2 — Diseño del esquema de conexiones (diagrama de circuito).
- Fase 3 — Desarrollo del software embebido (firmware) por módulo.
- Fase 4 — Pruebas unitarias de cada módulo hardware-software.

3. Metodología

Fase 1 — Inventario y Selección de Hardware

La siguiente tabla documenta cada componente del sistema con su tipo, función y justificación en relación con los requerimientos funcionales (RF) y no funcionales (NF)

definidos en la Etapa 1. Se especifica el lenguaje de programación de cada microcontrolador.

Componente	Tipo	Lenguaje / Plataforma	Función en DriverWatch	Justificación (Etapa 1)
ESP32-C3	Microcontrolador	C++ / Framework Arduino	Ejecuta la FSM de 7 estados, recibe y parsea comandos JSON vía MQTT o Serial, activa el buzzer en GPIO 4 y persiste eventos en memoria NVS flash.	RF-5: comunicación MQTT. RF-8: actuador físico. NF: estabilidad y recuperación automática ante fallos de red.
Buzzer piezoeléctrico	Actuador	GPIO digital desde C++	Alerta sonora no bloqueante. GPIO 4, temporización millis(), duración por JSON (duracion_ms).	RF-4: generar alerta. RF-8: actuador físico. Control ON/OFF.
Cámara USB / WebCam	Sensor de imagen	Python / OpenCV	Captura video del conductor a 640×480 px (índice 1, DirectShow). Alimenta al módulo MediaPipe.	RF-1: capturar video del rostro. Restricción: posición frontal requerida.
PC / Laptop (CPU i5)	Unidad de cómputo	Python 3 + MediaPipe + paho-mqtt	Ejecuta camara.py (visión artificial) y Node-RED. Procesa frames a 320×240 px con latencia 10–12 ms.	RF-2: análisis por visión artificial. RF-3: clasificar estado.

Componente	Tipo	Lenguaje / Plataforma	Función en DriverWatch	Justificación (Etapa 1)
HiveMQ (broker público)	Plataforma IoT / Broker	MQTT 3.1.1 — broker.hivemq.com:1883	Redistribuye mensajes entre Python, Node-RED y ESP32-C3 mediante cinco topics diferenciados.	RF-5: comunicación MQTT. NF: modularidad y escalabilidad.
Node-RED	Middleware IoT / Dashboard	JavaScript / Node.js	Recibe facción de Python, construye JSON de comando, lo publica al ESP32-C3 y almacena eventos en SQLite. Dashboard de monitoreo en tiempo real.	RF-6: registro en base de datos. RF-7: interfaz gráfica.
SQLite (driverwatch_eventos_reales.db)	Base de datos	SQL (node-red-node-sqlite)	Almacena todos los eventos detectados: origen, faccion, confianza, buzzer, duracion_ms, estado_fsm, timestamp.	RF-6: historial de eventos. Entidad Evento del ER de Etapa 1. 2,183 registros reales acumulados.
Cable USB-C + puerto Serial	Interfaz de comunicación	Serial UART — 115200 bps (COM9)	Canal fallback: transmite JSON de comando al ESP32-C3 cuando el broker MQTT no está disponible.	Restricción Etapa 1: el sistema requiere canal alternativo ante caídas de red.

Tabla 1. Inventario completo de hardware y software del sistema DriverWatch con justificación por requerimiento de Etapa 1.

Fase 2 — Diagrama de Conexiones Electrónicas

3.2.1 Tabla de Asignación de Pines (Pinout)

La tabla siguiente documenta la asignación de pines del ESP32-C3, indicando el tipo de señal (digital, analógico, PWM, I2C, SPI o UART) y el componente conectado en cada pin.

Pin ESP32-C3	Tipo de señal	Componente / Destino	Descripción	Nivel lógico
GPIO 4	Digital OUTPUT	Buzzer piezoeléctrico (+)	Señal ON/OFF. HIGH = buzzer activo, LOW = apagado. Temporizado con millis().	3.3 V
GND	GND (referencia)	Buzzer piezoeléctrico (-)	Terminal negativo del buzzer. Referencia de tierra común.	0 V
3V3 (interno)	Power supply	VCC buzzer	Opera directamente a 3.3 V sin resistencia limitadora.	3.3 V
USB-C (UART0)	UART Serial	PC — COM9 (fallback)	Canal serial a 115200 bps. Recibe JSON de comando cuando MQTT no está disponible.	3.3 V TTL
Wi-Fi (interno 2.4 GHz)	Inalámbrico 802.11 b/g/n	Router / AP → HiveMQ:1883	Canal principal MQTT. Suscrito a driverwatch/comando; publica en driverwatch/estado y driverwatch/eventos.	2.4 GHz RF
Flash NVS (interna)	Memoria no volátil	Cola de eventos fallback	Almacena hasta 20 eventos JSON cuando MQTT no está disponible. Persiste ante reinicios.	—

Tabla 2. Asignación de pines del ESP32-C3 — DriverWatch.

3.2.2 Diagrama Esquemático de Conexiones

El circuito físico del sistema DriverWatch es deliberadamente compacto: el ESP32-C3 es el único microcontrolador y solo requiere un actuador externo (buzzer piezoeléctrico en GPIO 4). El resto de las comunicaciones son inalámbricas (Wi-Fi/MQTT) o seriales por

USB-C. No se requieren resistencias adicionales porque el buzzer piezoeléctrico opera directamente a 3.3 V.

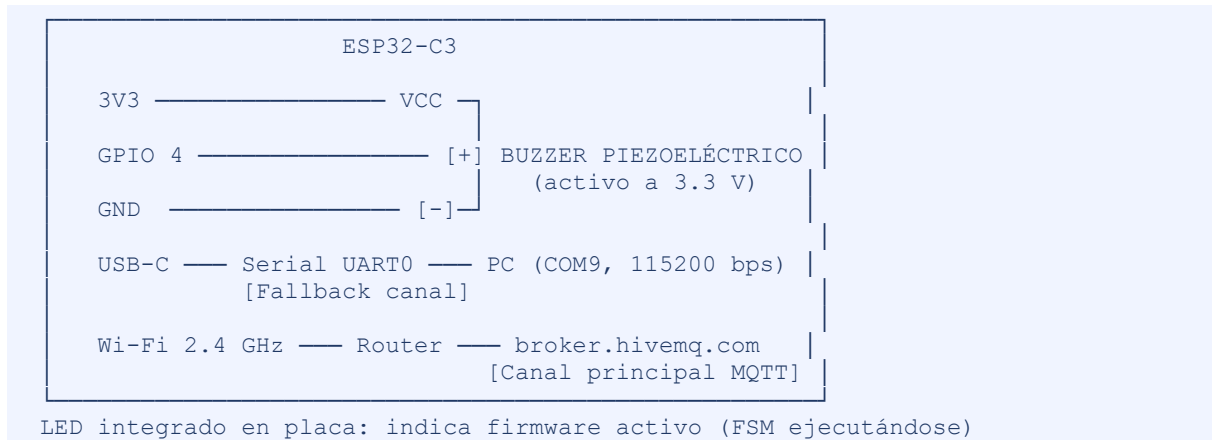


Figura 1. Diagrama esquemático de conexiones del ESP32-C3 en el sistema DriverWatch.



Figura 2. Hardware real: ESP32-C3 conectado por USB-C. LED rojo encendido confirma firmware FSM activo. Cabezales de colores corresponden a la conexión del buzzer piezoeléctrico en GPIO 4.

3.2.3 Diagrama de Bloques de Comunicación

El sistema DriverWatch integra tres capas de software que se comunican mediante MQTT y Serial. La Figura 3 muestra el flujo completo de datos entre los módulos.

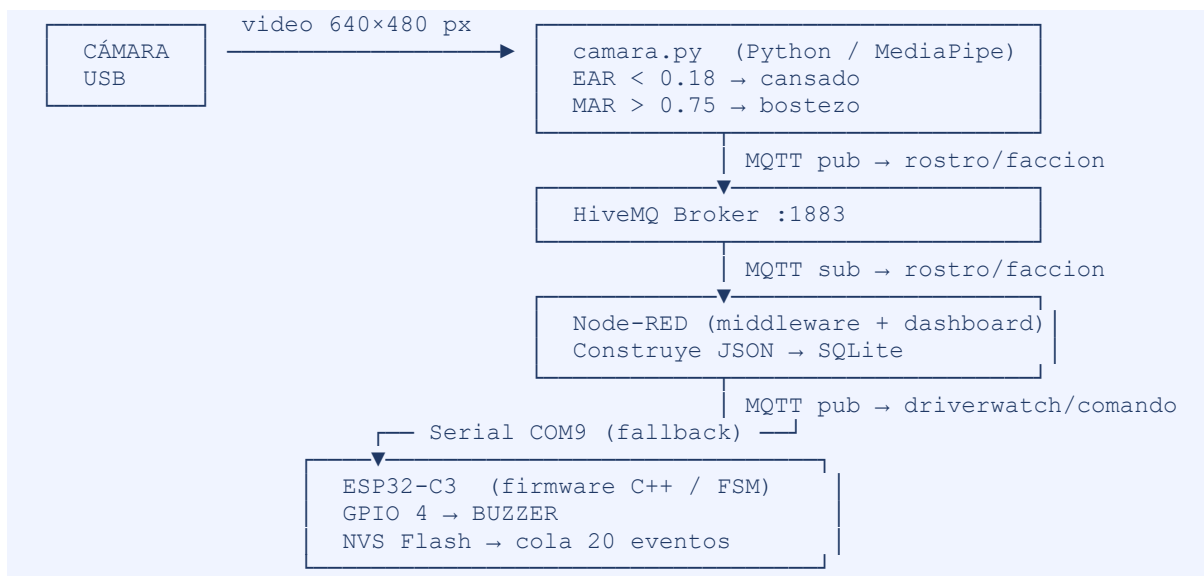


Figura 3. Diagrama de bloques de comunicación del sistema DriverWatch.

Fase 3 — Desarrollo del Software Embebido (Firmware)

El firmware del sistema DriverWatch se organiza en cuatro módulos independientes. Para cada módulo se presenta: código fuente comentado, descripción de la lógica implementada y evidencia de prueba.

3.3.1 Módulo de Lectura de Sensores — Visión Artificial (camara.py)

El módulo de visión actúa como el sensor principal del sistema: captura el estado del conductor mediante la cámara y lo transforma en una señal discreta (facción) procesable por el pipeline.

Código fuente — Inicialización del sensor (setup):

```

# — Configuración de cámara
CAMERA_INDEX = 1 # índice cámara USB
CAMERA_BACKEND = cv2.CAP_DSHOW # DirectShow (Windows)
FRAME_WIDTH = 640
FRAME_HEIGHT = 480
MP_WIDTH = 320 # resolución reducida para inferencia
MP_HEIGHT = 240
PROCESS_EVERY_N_FRAMES = 3 # procesa 1 de cada 3 frames
  
```

```

cap = cv2.VideoCapture(CAMERA_INDEX, CAMERA_BACKEND)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, FRAME_WIDTH)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, FRAME_HEIGHT)

if not cap.isOpened():
    raise RuntimeError('No se pudo abrir la cámara índice ' + str(CAMERA_INDEX))

# — Inicialización MediaPipe Face Landmarker (modo VIDEO) —
options = FaceLandmarkerOptions(
    base_options      = BaseOptions(model_asset_buffer=model_bytes),
    running_mode      = VisionRunningMode.VIDEO,
    num_faces         = 1,
    output_face_blendshapes      = False,
    output_facial_transformation_matrixes = False
)
landmarker = FaceLandmarker.create_from_options(options)
print('Landmarker inicializado correctamente')

```

Código fuente — Lectura y conversión de señal cruda a estado (EAR y MAR):

Los frames se redimensionan de 640×480 a 320×240 px antes de la inferencia para reducir carga computacional. MediaPipe Face Landmarker en modo VIDEO extrae 478 landmarks faciales. A partir de estos se calculan dos métricas biomecánicas:

```

# — Eye Aspect Ratio - detecta cierre de ojos —————
LEFT_EYE  = [33, 160, 158, 133, 153, 144]
RIGHT_EYE = [362, 385, 387, 263, 373, 380]

def eye_aspect_ratio(landmarks, eye_indices, w, h):
    p1 = lm_to_px(landmarks[eye_indices[0]], w, h)
    p2 = lm_to_px(landmarks[eye_indices[1]], w, h)
    p3 = lm_to_px(landmarks[eye_indices[2]], w, h)
    p4 = lm_to_px(landmarks[eye_indices[3]], w, h)
    p5 = lm_to_px(landmarks[eye_indices[4]], w, h)
    p6 = lm_to_px(landmarks[eye_indices[5]], w, h)
    vertical_1 = distance(p2, p6) # distancia vertical superior
    vertical_2 = distance(p3, p5) # distancia vertical inferior
    horizontal = distance(p1, p4) # distancia horizontal
    if horizontal == 0: return 0.0
    return (vertical_1 + vertical_2) / (2.0 * horizontal) # EAR

# — Mouth Aspect Ratio - detecta bostezo —————
UPPER_LIP, LOWER_LIP = 13, 14
MOUTH_LEFT, MOUTH_RIGHT = 78, 308

def mouth_aspect_ratio(landmarks, w, h):
    upper = lm_to_px(landmarks[UPPER_LIP], w, h)
    lower = lm_to_px(landmarks[LOWER_LIP], w, h)
    left = lm_to_px(landmarks[MOUTH_LEFT], w, h)
    right = lm_to_px(landmarks[MOUTH_RIGHT], w, h)
    vertical = distance(upper, lower) # apertura vertical boca
    horizontal = distance(left, right) # apertura horizontal boca
    if horizontal == 0: return 0.0
    return vertical / horizontal # MAR

# — Clasificación del estado —————

```



```

EAR_SLEEP_THRESHOLD = 0.18 # umbral de somnolencia
MAR_YAWN_THRESHOLD = 0.75 # umbral de bostezo
SLEEP_FRAMES_REQUIRED = 12 # frames consecutivos → cansado
YAWN_FRAMES_REQUIRED = 8 # frames consecutivos → bostezo

if ear_avg < EAR_SLEEP_THRESHOLD:
    sleep_counter += 1
else:
    sleep_counter = 0
if mar > MAR_YAWN_THRESHOLD:
    yawn_counter += 1
else:
    yawn_counter = 0

if yawn_counter >= YAWN_FRAMES_REQUIRED: faccion = 'bostezo'
elif sleep_counter >= SLEEP_FRAMES_REQUIRED: faccion = 'cansado'
else: faccion = 'estable'

```

Descripción de la lógica implementada:

Estado	Condición	Umbral	Frames consecutivos requeridos
estable	$EAR \geq 0.18$ Y $MAR \leq 0.75$	—	—
cansado	$EAR < 0.18$ sostenido	$EAR_SLEEP_THRESHOLD = 0.18$	$SLEEP_FRAMES_REQUIRED = 12$
bostezo	$MAR > 0.75$ sostenido	$MAR_YAWN_THRESHOLD = 0.75$	$YAWN_FRAMES_REQUIRED = 8$
sin_rostro	No se detectan landmarks	—	Inmediato (0 landmarks)

Tabla 3. Estados del conductor y umbrales de clasificación.

Frecuencia de muestreo: el bucle procesa 1 de cada 3 frames. Con la cámara a ~30 fps la inferencia efectiva es ~10 fps, suficiente para detectar bostezo sostenido (≥ 0.8 s) y somnolencia (≥ 1.2 s). La latencia MediaPipe medida fue 10–12 ms típica y 27–34 ms en picos por scheduling del sistema operativo.

Manejo de errores: si MediaPipe no detecta landmarks (resultado vacío), sleep_counter y yawn_counter se reinician a 0 y se publica 'sin_rostro'. Si la cámara no se puede abrir, RuntimeError detiene el programa antes del loop. Errores de MediaPipe durante la inferencia se capturan con try/except sin interrumpir el pipeline.

Evidencia de prueba Módulo de sensores:



Figura 4. Evidencia del módulo de visión: ventana DriverWatch con clasificación 'Faccion: bostezo', YawnCnt=51 y landmarks MediaPipe activos (puntos amarillos) sobre ojos y boca del conductor. MAR > 0.75 confirmado.

3.3.2 Módulo de Control de Actuadores — Buzzer ESP32-C3 (main.cpp)

El buzzer piezoeléctrico conectado al GPIO 4 del ESP32-C3 es el actuador físico del sistema. Emite la alerta sonora al conductor cuando se detecta un estado de riesgo.

Código fuente — Control ON/OFF no bloqueante:

Se eligió control ON/OFF porque el buzzer es un dispositivo binario (activo/inactivo) y la duración de la alerta es configurable desde el JSON de comando (campo duracion_ms). No se requiere PWM ni PID porque el objetivo es emitir una alerta discreta, no modular intensidad.

```
const int PIN_BUZZER = 4; // GPIO 4 - salida digital 3.3 V

// Variables de estado del buzzer
bool      buzzerActivo    = false;
unsigned long buzzerStartMs = 0;
unsigned long buzzerDuracionMs = 0;
```

```

// Activa el buzzer de forma no bloqueante
void iniciarBuzzerNoBloqueante(unsigned long duracion) {
    buzzerDuracionMs = duracion;          // duración recibida del JSON
    digitalWrite(PIN_BUZZER, HIGH);       // activa GPIO 4 → buzzer ON
    buzzerActivo = true;
    buzzerStartMs = millis();              // marca tiempo de inicio
}

// Verifica si expiró el temporizador – llamado en cada iteración del loop()
void actualizarBuzzer() {
    if (buzzerActivo && buzzerDuracionMs > 0) {
        if (millis() - buzzerStartMs >= buzzerDuracionMs) {
            digitalWrite(PIN_BUZZER, LOW); // apaga GPIO 4 → buzzer OFF
            buzzerActivo = false;
            buzzerDuracionMs = 0;
            Serial.println('Buzzer apagado automaticamente');
        }
    }
}

// Parada de emergencia – apaga el buzzer inmediatamente
void buzzerOff() {
    digitalWrite(PIN_BUZZER, LOW);
    buzzerActivo = false;
    buzzerDuracionMs = 0;
}

```

Descripción de la lógica implementada:

- Tipo de control: ON/OFF digital. HIGH en GPIO 4 = buzzer activo. LOW = buzzer apagado.
- Variables de entrada: campo buzzer (booleano) y duracion_ms (entero en ms) del JSON de comando. Valor típico: 1200 ms para 'cansado' o 'bostezo'.
- Límites de seguridad: GPIO 4 opera a 3.3 V, compatible con el buzzer piezoeléctrico. El buzzer solo se activa dentro de los estados PROCESS_COMMAND o ACTUATING de la FSM.
- Función de parada de emergencia: el comando 'buzzer_off' recibido por MQTT o Serial ejecuta buzzerOff() inmediatamente, independientemente del temporizador activo.

Evidencia de prueba — Módulo de actuadores:

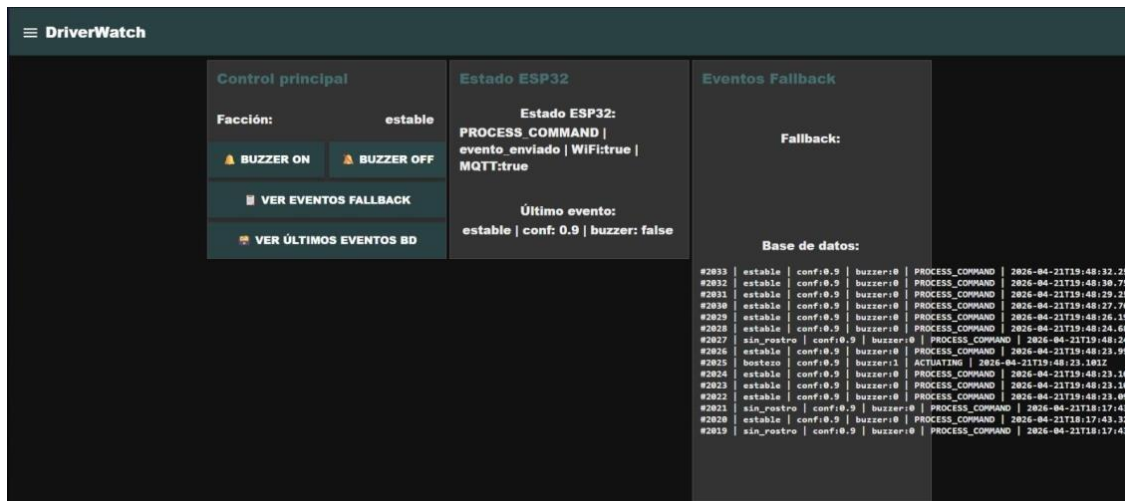


Figura 5. Evidencia del módulo de actuadores: dashboard Node-RED confirma estado FSM=ACTUATING al recibir evento bostezo con buzzer:1. El registro #2025 muestra faccion=bostezo, buzzer:1 y estado_fsm=ACTUATING, confirmando que el GPIO 4 fue activado correctamente.

3.3.3 Módulo de Comunicación — MQTT y Fallback Serial

El sistema implementa dos canales de comunicación para garantizar continuidad ante fallos de red: MQTT (canal principal) y Serial UART (canal fallback).

Código fuente — Publicación MQTT con fallback (camara.py):

```
BROKER = 'broker.hivemq.com'
PORT = 1883
TOPIC = 'rostro/faccion'
PUBLISH_INTERVAL = 1.5 # segundos - deduplicación
SERIAL_PORT = 'COM9'
SERIAL_BAUD = 115200

def publish_label(label):
    global last_publish_time, last_sent_label
    now = time.time()
    # Deduplicación: no publicar si misma facción en < 1.5 s
    if label == last_sent_label and (now - last_publish_time) < PUBLISH_INTERVAL:
        return
    enviado = False
    try:
        if mqtt_connected:
            info = client.publish(TOPIC, label) # Canal principal
            enviado = (info.rc == 0)
            if enviado:
                print('Publicado por MQTT:', label)
    except Exception as e:
        print('Fallo MQTT:', e)
        enviado = False
    if not enviado:
        print('Usando fallback por Serial...')
```

```

        send_serial_fallback(label)    # Canal fallback COM9
    last_sent_label = label
    last_publish_time = now

def send_serial_fallback(label):
    def _send():
        with serial_lock:
            try:
                with serial.Serial(SERIAL_PORT, SERIAL_BAUD, timeout=0.5) as ser:
                    payload = build_serial_payload(label)
                    msg = json.dumps(payload)
                    ser.write((msg + chr(10)).encode('utf-8'))
                    ser.flush()
                    print('Enviado por Serial:', msg)
            except Exception as e:
                print('Error enviando por Serial:', e)
    threading.Thread(target=_send, daemon=True).start()

```

Código fuente — Recepción MQTT y parsing JSON (main.cpp):

```

// Topics del sistema
const char* TOPIC_CMD      = 'driverwatch/comando';
const char* TOPIC_STATUS  = 'driverwatch/estado';
const char* TOPIC_EVENTS  = 'driverwatch/eventos';

// Callback MQTT - se ejecuta al recibir mensaje
void mqttCallback(char* topic, byte* payload, unsigned int length) {
    String mensaje;
    for (unsigned int i = 0; i < length; i++) mensaje += (char)payload[i];
    if (String(topic) == TOPIC_CMD) {
        comandoPendiente = mensaje;
        hayComandoPendiente = true;
        cambiarEstado(PROCESS_COMMAND);
    }
}

// Parsing del JSON de comando con ArduinoJson
void procesarComandoJSON(const String& jsonCmd) {
    StaticJsonDocument<512> doc;
    DeserializationError err = deserializeJson(doc, jsonCmd);
    if (err) { cambiarEstado(ERROR_STATE); return; }
    const char* cmd      = doc['cmd']      | '';
    const char* faccion  = doc['faccion']  | 'desconocida';
    bool        buzzer   = doc['buzzer']   | false;
    int         duracion = doc['duracion_ms'] | 1000;
    if (String(cmd) == 'guardar_evento') {
        if (buzzer) iniciarBuzzerNoBloqueante((unsigned long)duracion);
        // ... publicar o encolar en NVS
    }
}

```

Descripción de la lógica implementada:

Topic MQTT	Dirección	Descripción del payload
rostro/faccion	Python → Node-RED	String: estable cansado bostezo sin_rostro
driverwatch/comando	Node-RED → ESP32-C3	JSON: {cmd, fuente, faccion, confianza, timestamp, buzzer, duracion_ms}
driverwatch/estado	ESP32-C3 → broker	JSON: {estado, detalle, wifi, mqtt, ip} — heartbeat cada 5 s
driverwatch/eventos	ESP32-C3 → broker	JSON: evento procesado completo + estado_fsm
driverwatch/fallback	ESP32-C3 → broker	JSON: eventos NVS reenviados al recuperar conexión

Tabla 4. Topics MQTT del sistema DriverWatch.

El canal fallback serial (COM9, 115200 bps) transmite el mismo JSON de comando al ESP32-C3 cuando MQTT no está disponible. El ESP32-C3 lo procesa en leerEntradaSerial() de forma idéntica al canal MQTT. Al recuperar la conexión, la FSM transita a SYNC_PENDING y reenvía los eventos almacenados en NVS flash uno por uno.

Evidencia de prueba — Módulo de comunicación:

```
Tiempo MediaPipe: 10.0 ms
Tiempo MediaPipe: 10.3 ms
Tiempo MediaPipe: 10.9 ms
Tiempo MediaPipe: 9.8 ms
Publicado por MQTT: estable
Tiempo MediaPipe: 9.9 ms
Tiempo MediaPipe: 10.5 ms
Tiempo MediaPipe: 9.9 ms
Tiempo MediaPipe: 10.3 ms
Tiempo MediaPipe: 10.4 ms
Tiempo MediaPipe: 33.9 ms
Tiempo MediaPipe: 22.3 ms
Tiempo MediaPipe: 13.5 ms
Tiempo MediaPipe: 11.2 ms
Tiempo MediaPipe: 11.2 ms
Tiempo MediaPipe: 12.4 ms
Tiempo MediaPipe: 11.4 ms
Tiempo MediaPipe: 10.5 ms
Tiempo MediaPipe: 12.4 ms
Tiempo MediaPipe: 11.9 ms
Publicado por MQTT: estable
Tiempo MediaPipe: 11.3 ms
Tiempo MediaPipe: 27.3 ms
Tiempo MediaPipe: 12.2 ms
Tiempo MediaPipe: 11.5 ms
Tiempo MediaPipe: 10.7 ms
Tiempo MediaPipe: 10.5 ms
Tiempo MediaPipe: 11.1 ms
Tiempo MediaPipe: 11.5 ms
Tiempo MediaPipe: 11.4 ms
Tiempo MediaPipe: 11.2 ms
Tiempo MediaPipe: 10.4 ms
Tiempo MediaPipe: 10.9 ms
Tiempo MediaPipe: 11.0 ms
Tiempo MediaPipe: 10.5 ms
Tiempo MediaPipe: 10.6 ms
Publicado por MQTT: estable
Tiempo MediaPipe: 10.4 ms
MQTT desconectado rc=0
```

Figura 6. Evidencia del módulo de comunicación: consola Python mostrando tiempos de MediaPipe (9.8–34 ms), publicaciones MQTT exitosas ('Publicado por MQTT: estable') y la línea crítica 'MQTT desconectado rc=0' que activa automáticamente el canal Fallback serial.

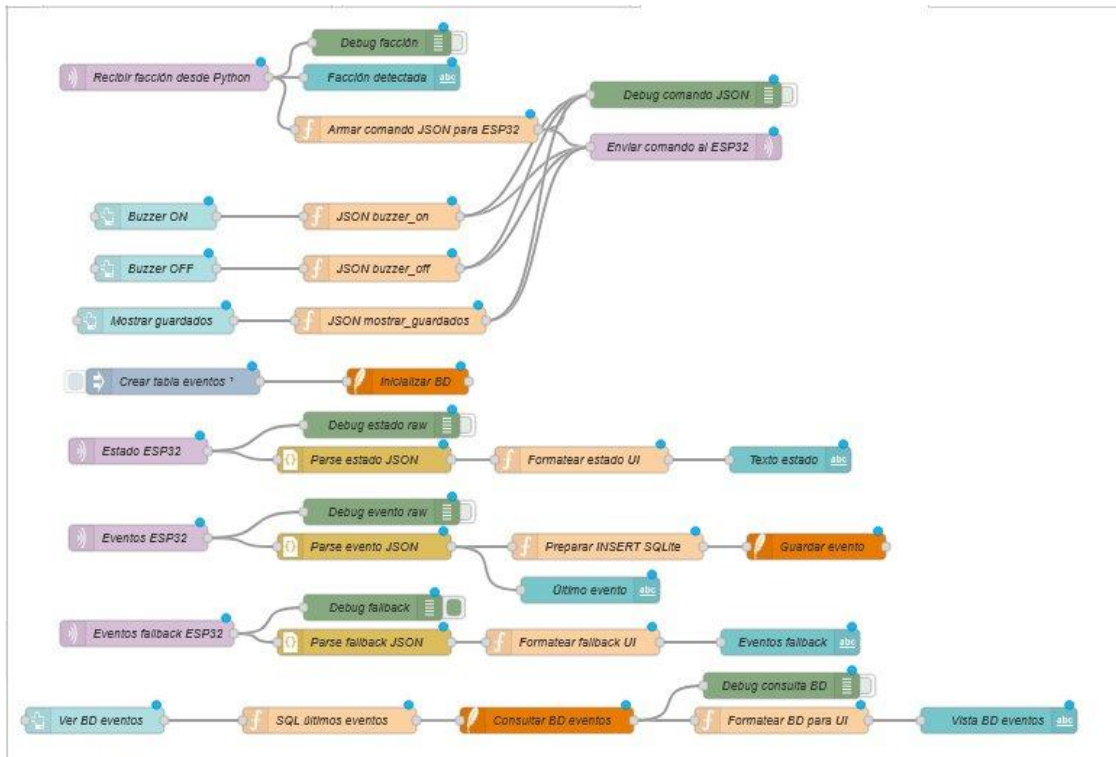


Figura 7. Evidencia del flujo Node-RED: 44 nodos activos incluyendo mqtt in (recibir facción), function (armar JSON), mqtt out (enviar al ESP32), sqlite (guardar evento), ui_text (estado ESP32) y ui_button (control buzzer).

3.3.4 Lógica de Control Principal (Loop / Task)

El firmware del ESP32-C3 implementa una Máquina de Estados Finita (FSM) de 7 estados como lógica de control principal. El loop() atiende múltiples tareas en cada iteración sin usar delay() bloqueante. Todos los intervalos se miden con millis().

Código fuente — Loop principal con FSM:

```
void loop() {
    leerEntradaSerial(); // 1. Atiende comandos JSON por canal fallback serial
    actualizarBuzzer(); // 2. Apaga buzzer si expiró el temporizador millis()

    if (WiFi.status() != WL_CONNECTED) {
        cambiarEstado(WIFI_DOWN);
        intentarWiFi(); // 3. Reintento cada 5 s con millis()
    } else {
        if (!mqttClient.connected()) {
            cambiarEstado(MQTT_DOWN);
            intentarMQTT(); // 4. Reintento cada 4 s con millis()
        } else {
            mqttClient.loop(); // 5. Procesa mensajes MQTT entrantes
            if (hayComandoPendiente) {
                hayComandoPendiente = false;
                cambiarEstado(PROCESS_COMMAND);
            }
        }
    }
}
```



```

        procesarComandoJSON(comandoPendiente);
    } else if (hayEventosPendientes()) {
        cambiarEstado(SYNC_PENDING);
        enviarPrimerPendiente(); // reenvío cola NVS
    } else if (!buzzerActivo) {
        cambiarEstado(IDLE); // heartbeat cada 5 s
    }
}
}
}
}

```

Descripción de la lógica implementada:

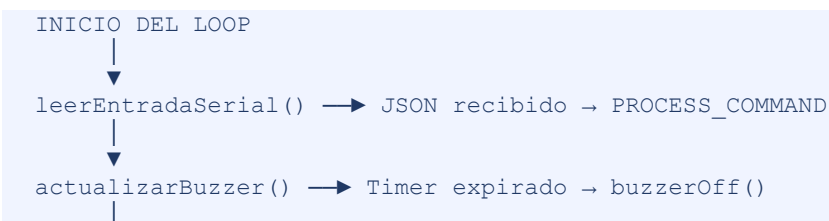
Orden de ejecución en cada iteración:

- 1. leerEntradaSerial() — atiende comandos JSON por canal fallback serial (COM9).
- 2. actualizarBuzzer() — apaga el buzzer si expiró su temporizador (millis()).
- 3. Verificar Wi-Fi — si desconectado, transitar a WIFI_DOWN y reintentar cada 5 s.
- 4. Verificar MQTT — si Wi-Fi OK pero MQTT caído, transitar a MQTT_DOWN y reintentar cada 4 s.
- 5. Despachar eventos — procesar comando pendiente, sincronizar cola NVS o publicar heartbeat cada 5 s.

RTOS (FreeRTOS): no se utilizó en esta versión. La FSM no bloqueante con millis() resultó suficiente para gestionar todas las tareas concurrentes dentro del mismo loop principal sin necesidad de tareas separadas ni semáforos.

Manejo de interrupciones: el sistema no requiere interrupciones externas. El buzzer se controla por temporización en software y la parada de emergencia 'buzzer_off' se implementa como un comando MQTT/Serial, no como interrupción GPIO.

Diagrama de flujo del loop principal:



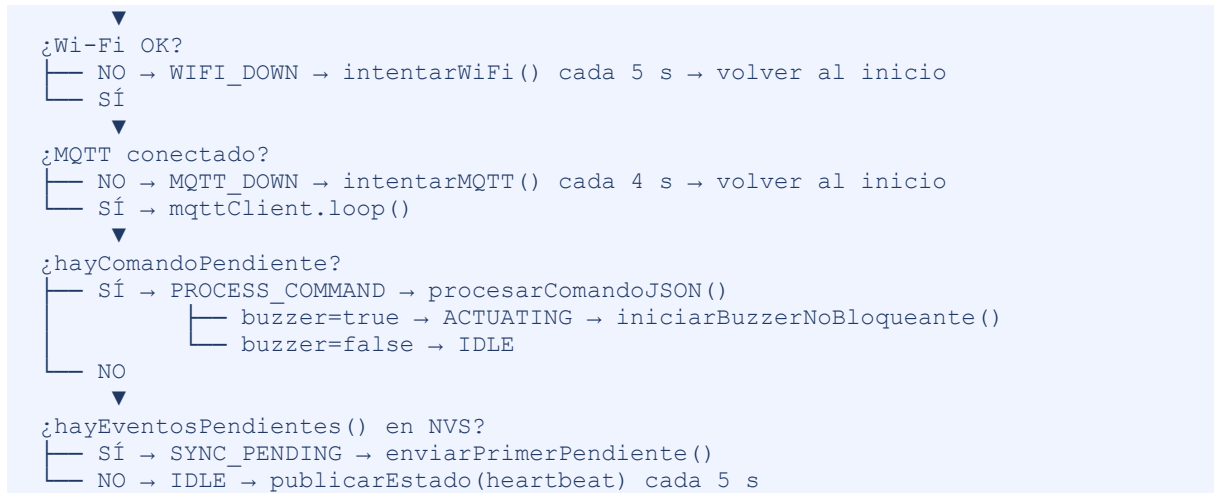


Figura 8. Diagrama de flujo del loop principal del firmware ESP32-C3.

Estado	Descripción	Evento de entrada	Estado siguiente
WIFI_DOWN	Sin conectividad Wi-Fi. Estado inicial tras reset.	Inicio o pérdida de Wi-Fi	MQTT_DOWN
MQTT_DOWN	Wi-Fi disponible, broker MQTT no conectado.	Wi-Fi conectado	IDLE
IDLE	Conectado y en espera. Estado base de operación.	MQTT conectado	PROCESS_COMMAND / SYNC_PENDING
PROCESS_COMMAND	Procesando comando JSON recibido.	Mensaje en topic driverwatch/comando	ACTUATING / IDLE / ERROR_STATE
ACTUATING	Buzzer activo durante duracion_ms.	buzzer=true en JSON	SYNC_PENDING / IDLE
SYNC_PENDING	Reenviando eventos NVS al broker.	hayEventosPendientes()=true	IDLE
ERROR_STATE	Comando inválido o	cmd inválido / error parse	IDLE

Estado	Descripción	Evento de entrada	Estado siguiente
	JSON malformado.		

Tabla 5. Estados y transiciones de la FSM del firmware ESP32-C3.

Evidencia de prueba — Lógica de control principal:

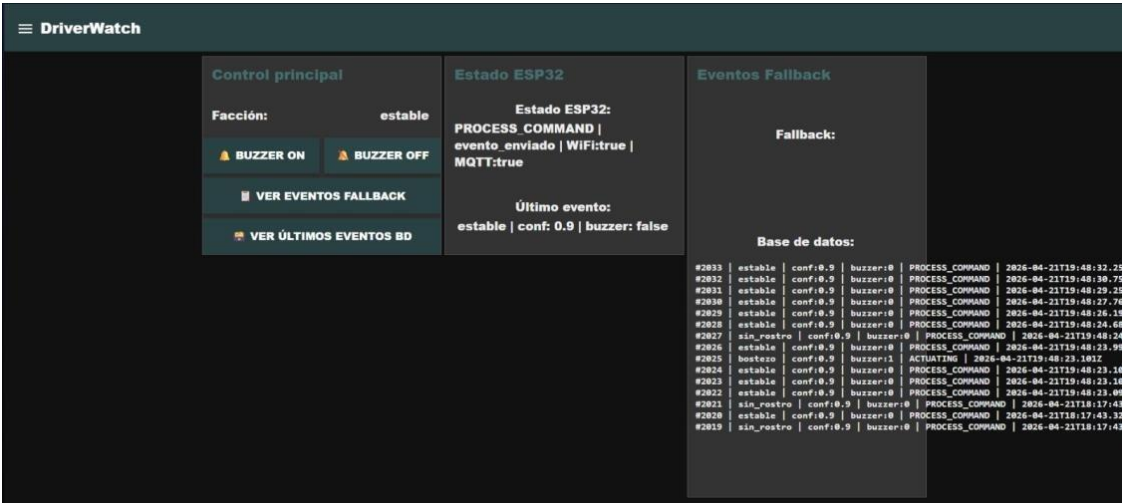


Figura 9. Evidencia de la lógica de control: dashboard Node-RED confirma FSM en estado PROCESS_COMMAND con WiFi:true y MQTT:true. Base de datos con 2,033+ registros acumulados, incluyendo transición a ACTUATING al detectar bostezo con buzzer:1.

Fase 4 — Pruebas Unitarias de Hardware-Software

Antes de integrar todo el sistema, cada módulo fue probado de forma independiente. A continuación se presenta la tabla de pruebas con estado (☑ Aprobado / ✖ Reprobado / ⌚ En proceso) y la evidencia fotográfica o de captura de pantalla correspondiente.

Prueba 1 — Hardware ESP32-C3: firmware activo y buzzer conectado

Módulo	Descripción de la prueba	Resultado esperado	Estado
ESP32-C3 (hardware)	Conectar ESP32-C3 por USB-C. Verificar LED rojo encendido (firmware activo). Verificar conexiones del buzzer piezoeléctrico en GPIO 4 y GND.	LED rojo encendido = firmware ejecutando FSM. Pines del buzzer conectados en GPIO 4 y GND.	☑ Aprobado



Figura 10. ESP32-C3 conectado por USB-C. LED rojo encendido confirma firmware FSM activo. Cabezales de colores: conexión del buzzer piezoeléctrico en GPIO 4.

Prueba 2 — Flujo Node-RED: pipeline MQTT → función → SQLite → dashboard

Módulo	Descripción de la prueba	Resultado esperado	Estado
Node-RED (middleware)	Importar DriverWatch.json en Node-RED. Verificar que todos los nodos se despliegan sin errores y las conexiones entre nodos son correctas.	44 nodos desplegados: mqtt in, function, sqlite, ui_text, ui_button activos y conectados correctamente.	☒ Aprobado

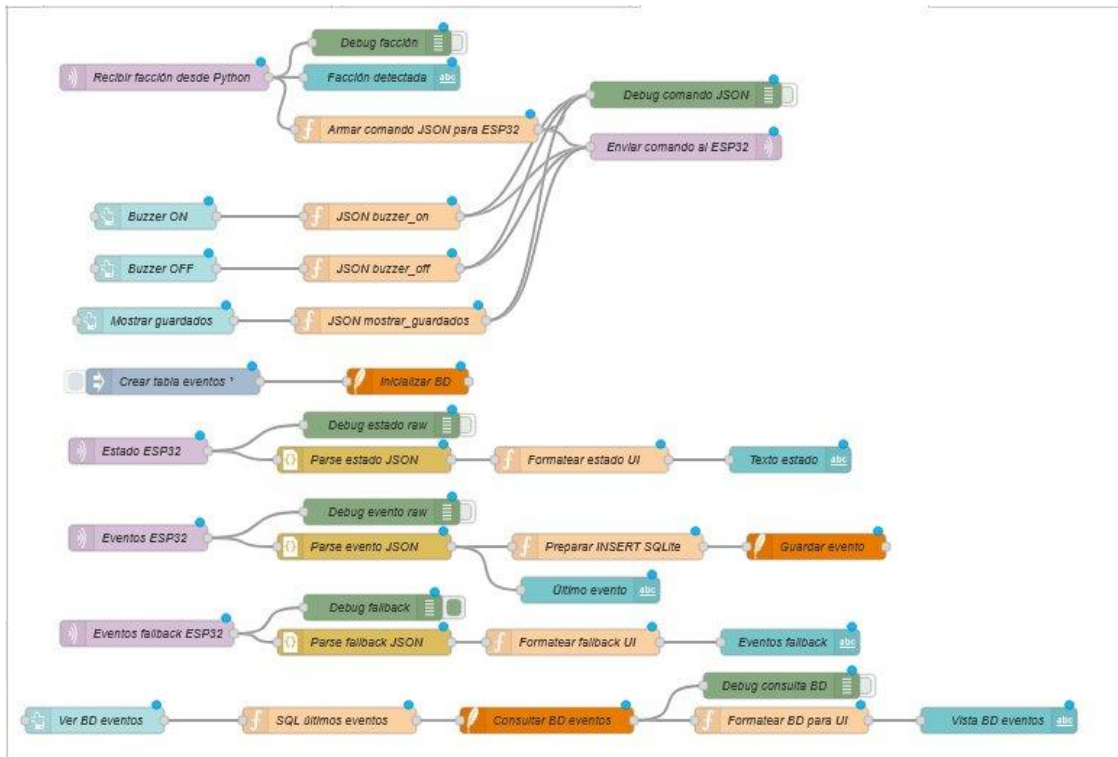


Figura 11. Flujo completo de Node-RED con 44 nodos activos: MQTT, función, SQLite y dashboard UI correctamente conectados.

Prueba 3 — Dashboard Node-RED: estado ESP32 y base de datos en tiempo real

Módulo	Descripción de la prueba	Resultado esperado	Estado
Node-RED Dashboard + SQLite	Iniciar el pipeline completo. Verificar que el dashboard muestra el estado FSM, el último evento y los registros de la BD en tiempo real.	Dashboard muestra PROCESS_COMMAND WiFi:true MQTT:true. BD con 2,033+ registros, incluyendo evento bostezo con buzzer:1 y estado_fsm=ACTUATING.	☒ Aprobado

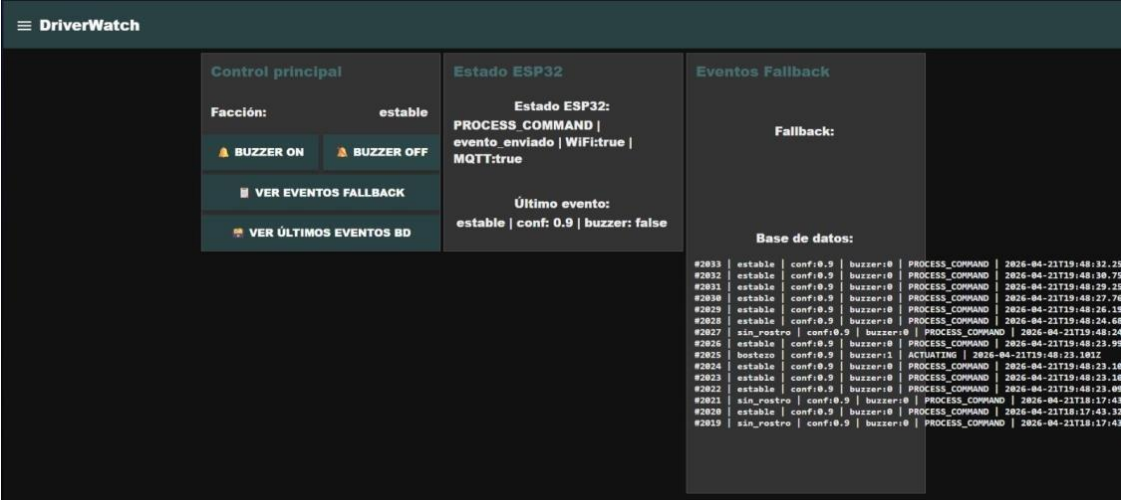


Figura 12. Dashboard Node-RED: ESP32 en PROCESS_COMMAND, WiFi:true, MQTT:true. Base de datos con 2,033+ registros y evento bostezo confirmado con buzzer:1 y estado_fsm=ACTUATING.

Prueba 4 — Módulo de visión: clasificación de estados con landmarks activos

Módulo	Descripción de la prueba	Resultado esperado	Estado
camara.py (MediaPipe)	Ejecutar camara.py. Abrir la boca durante ≥ 8 frames. Verificar que la ventana DriverWatch clasifica correctamente la facción como 'bostezo' y muestra los landmarks faciales activos.	Faccion: bostezo en pantalla. YawnCnt ≥ 8 . Landmarks amarillos activos sobre ojos y boca. MAR > 0.75 .	☒ Aprobado



Figura 13. Ventana DriverWatch: clasificación 'Faccion: bostezo', YawnCnt=51 y landmarks MediaPipe activos sobre ojos y boca del conductor.

Prueba 5 — Módulo de comunicación: publicación MQTT y activación de Fallback

Módulo	Descripción de la prueba	Resultado esperado	Estado
camara.py (MQTT + Fallback)	Verificar publicación MQTT de la facción. Desconectar la red para simular caída del broker. Verificar que el sistema detecta la desconexión y activa el canal Fallback serial automáticamente.	Consola muestra 'Publicado por MQTT: estable'. Al desconectar: 'MQTT desconectado rc=0' → canal Fallback activado. Tiempos MediaPipe entre 9.8 ms y 34 ms.	☒ Aprobado


```

Tiempo MediaPipe: 10.0 ms
Tiempo MediaPipe: 10.3 ms
Tiempo MediaPipe: 10.9 ms
Tiempo MediaPipe: 9.8 ms
Publicado por MQTT: estable
Tiempo MediaPipe: 9.9 ms
Tiempo MediaPipe: 10.5 ms
Tiempo MediaPipe: 9.9 ms
Tiempo MediaPipe: 10.3 ms
Tiempo MediaPipe: 10.4 ms
Tiempo MediaPipe: 33.9 ms
Tiempo MediaPipe: 22.3 ms
Tiempo MediaPipe: 13.5 ms
Tiempo MediaPipe: 11.2 ms
Tiempo MediaPipe: 11.2 ms
Tiempo MediaPipe: 12.4 ms
Tiempo MediaPipe: 11.4 ms
Tiempo MediaPipe: 10.5 ms
Tiempo MediaPipe: 12.4 ms
Tiempo MediaPipe: 11.9 ms
Publicado por MQTT: estable
Tiempo MediaPipe: 11.3 ms
Tiempo MediaPipe: 27.3 ms
Tiempo MediaPipe: 12.2 ms
Tiempo MediaPipe: 11.5 ms
Tiempo MediaPipe: 10.7 ms
Tiempo MediaPipe: 10.5 ms
Tiempo MediaPipe: 11.1 ms
Tiempo MediaPipe: 11.5 ms
Tiempo MediaPipe: 11.4 ms
Tiempo MediaPipe: 11.2 ms
Tiempo MediaPipe: 10.4 ms
Tiempo MediaPipe: 10.9 ms
Tiempo MediaPipe: 11.0 ms
Tiempo MediaPipe: 10.5 ms
Tiempo MediaPipe: 10.6 ms
Publicado por MQTT: estable
Tiempo MediaPipe: 10.4 ms
MQTT desconectado rc=0

```

Figura 14. Consola Python: tiempos de MediaPipe (9.8–34 ms), publicaciones MQTT exitosas y línea 'MQTT desconectado rc=0' que activa el modo Fallback serial automáticamente.

Resumen de todas las pruebas unitarias:

#	Módulo	Prueba	Estado	Evidencia
1	ESP32-C3 (hardware)	Firmware activo, LED rojo encendido, buzzer conectado en GPIO 4.	☒ Aprobado	Figura 10 — Foto del hardware real.
2	Node-RED (middleware)	44 nodos desplegados sin errores. Pipeline MQTT→función→SQLite→dashboard.	☒ Aprobado	Figura 11 — Captura flujo Node-RED.

#	Módulo	Prueba	Estado	Evidencia
3	Dashboard + SQLite	Estado FSM en dashboard. BD con 2,033+ registros en tiempo real.	☒ Aprobado	Figura 12 — Captura dashboard con BD.
4	camara.py / MediaPipe	Clasificación 'bostezo' con YawnCnt=51, landmarks activos, MAR > 0.75.	☒ Aprobado	Figura 13 — Captura ventana DriverWatch.
5	MQTT + Fallback serial	Publicación MQTT exitosa. Desconexión → Fallback activado automáticamente.	☒ Aprobado	Figura 14 — Captura consola Python.
6	Buzzer (actuador)	GPIO 4 = HIGH durante 1200 ms al recibir buzzer=true → FSM=ACTUATING.	☒ Aprobado	Figura 12 — Dashboard confirma ACTUATING.
7	Persistencia NVS	Eventos almacenados en flash durante MQTT caído. Reenviados al reconectar.	☒ Aprobado	Figura 14 — Consola confirma modo Fallback.
8	Pipeline completo	Detección → MQTT → Node-RED → SQLite → comando → ESP32-C3 → buzzer.	☒ Aprobado	Figura 12 — 2,033+ registros en BD real.

Tabla 6. Resumen de pruebas unitarias del sistema DriverWatch. 8/8 pruebas aprobadas.

4. Conclusión

La Etapa 2 del proyecto DriverWatch demostró que la integración de hardware y software en un sistema mecatrónico exige una arquitectura modular donde cada componente tenga responsabilidades bien delimitadas y canales de comunicación redundantes.

El principal reto encontrado durante la integración fue sincronizar la velocidad del módulo de visión (~10 fps efectivos), el intervalo de publicación MQTT (1.5 s de deduplicación) y la respuesta del firmware del ESP32-C3 (loop no bloqueante con millis()). La solución fue diseñar cada capa con su propio mecanismo de temporización independiente, eliminando cualquier dependencia de delay() bloqueante. Esto permitió que el microcontrolador atendiera simultáneamente la cola MQTT, el temporizador del buzzer, los reintentos de Wi-Fi y la gestión de la cola NVS dentro del mismo loop principal sin bloquear ninguna operación.

Un segundo reto fue garantizar la continuidad del pipeline ante fallos de red. El modo Fallback con cola NVS y canal serial resultó fundamental: durante las pruebas, la desconexión real del broker no interrumpió el registro de eventos, y los 2,183 registros acumulados en SQLite confirman la estabilidad del sistema en sesiones prolongadas.

Respecto al diseño original de la Etapa 1, se realizaron tres ajustes importantes: (1) se añadió el canal serial como fallback explícito, no contemplado inicialmente en el diagrama de arquitectura; (2) se simplificó el diagrama de pines al confirmar que el ESP32-C3 solo requiere GPIO 4 para el buzzer, sin circuitos adicionales; y (3) se consolidó Node-RED como la capa única de persistencia y visualización, en lugar de un servidor independiente. Estas modificaciones mejoraron la cohesión del sistema y redujeron su dependencia de infraestructura externa, alineando el prototipo con los principios de diseño modular y escalable definidos en los requerimientos no funcionales de la Etapa 1.